

UNIVERSIDADE KIMPA VITA · Instituto Politécnico de Uíge

Licenciatura em Engenharia Informática · 3.º Ano

ROTEIRO DE APRESENTAÇÃO SGBD II

*Catálogo de Produtos Dinâmicos com Pesquisa Facetada
utilizando MongoDB — SGBD NoSQL*

-
1. Américo Malungo Sebastião Miguel
 2. Bengui Bena Pedro
 3. Egas Manuel Ribeiro
 4. Matondo Domingos Bunga
 5. Hermenegildo Wilson dos Santos Panzo

Docente: Moyo Kanivengidio · Uíge, Maio de 2026

Disciplina: Sistemas de Gestão de Bases de Dados II · 2025/2026

Abertura da Apresentação

■ AMÉRICO MALUNGO SEBASTIÃO MIGUEL

[Aguardar que o slide esteja visível. Postura direita, falar para a audiência.]

Bom dia, senhor Professor Kanivengidio e colegas.

Somos o Grupo [número do grupo] e vamos apresentar o nosso trabalho prático de SGBD II, que tem como tema: Modelação e Implementação de um Catálogo de Produtos Dinâmicos com Pesquisa Facetada, utilizando MongoDB como SGBD NoSQL.

O grupo é composto por cinco elementos: Américo Malungo, Bengui Bena, Egas Manuel, Matondo Domingos e Hermenegildo Wilson.

Antes de começarmos, deixem-me apresentar brevemente a estrutura da nossa apresentação.

Estrutura da Apresentação

■ AMÉRICO MALUNGO SEBASTIÃO MIGUEL (continua)

A nossa apresentação está organizada em cinco secções:

- 1. Primeiro, contextualizamos o Problema e o Domínio — o que é um catálogo de e-commerce de alta escala e quais os seus desafios.*
- 2. De seguida, fazemos a Justificação Tecnológica, onde aplicamos o Teorema CAP para fundamentar a escolha do MongoDB.*
- 3. Na terceira secção, apresentamos a Modelação de Dados com base no Query-Driven Design.*
- 4. Depois, mostramos a Implementação prática — Docker, dados gerados e as 6 queries avançadas.*
- 5. Por fim, apresentamos os Resultados e a Discussão crítica sobre escalabilidade e tolerância a falhas.*

Peço que guardem as questões para o final. Passamos a palavra ao Bengui para a secção 1.

Catálogo de Produtos de Alta Escala

■ BENGUI BENA PEDRO

Obrigado, Américo.

O nosso trabalho foca-se num catálogo de produtos para um e-commerce de alta escala. Imagine uma plataforma como a Amazon ou Jumia, com milhões de produtos de categorias completamente diferentes — smartphones, roupas, livros, utensílios de cozinha.

O problema central está aqui: cada categoria tem atributos completamente diferentes. Um smartphone tem RAM e câmara; uma camiseta tem tamanho, cor e material. Num banco relacional tradicional, isto forçar-nos-ia a criar múltiplas tabelas ou a usar o padrão EAV — Entity-Attribute-Value — que é extremamente ineficiente.

Os nossos requisitos de sistema são: performance sub-segundo nas buscas — o P99 tem de ser inferior a 500 milissegundos — alta disponibilidade 24 horas por dia, 7 dias por semana, e escalabilidade horizontal para suportar o crescimento dos dados.

No lado direito do slide, podem ver os padrões de acesso — ou seja, como o sistema vai ser utilizado. A busca facetada, que é a pesquisa com múltiplos filtros simultâneos, representa 60% do tráfego total, com 500 pedidos por segundo. É este padrão que vai guiar todas as nossas decisões de modelação.

Esta abordagem chama-se Query-Driven Design — primeiro definimos como vamos consultar os dados, e só depois desenhamos o modelo.

Teorema CAP — Escolha Arquitetural

■ BENGUI BENA PEDRO (continua)

Com os requisitos definidos, precisamos de escolher o SGBD adequado. Para isso, aplicamos o Teorema CAP de Brewer.

O Teorema CAP diz-nos que um sistema distribuído só pode garantir duas das três propriedades em simultâneo: Consistência, Disponibilidade, e Tolerância a Partições de rede.

Para o nosso caso — um e-commerce que não pode parar — escolhemos AP: Availability e Partition Tolerance. Porquê? Porque a disponibilidade é crítica. Se o sistema ficar em baixo, o negócio para. E a consistência eventual é aceitável — se um produto mostrar um preço desatualizado por alguns segundos, não causa dano real ao negócio.

Comparámos três alternativas: o Cassandra, também AP, mas com schema rígido, não consegue lidar nativamente com atributos dinâmicos por categoria; o Elasticsearch, excelente para full-text search, mas não é uma fonte de verdade primária; e o MongoDB, com schema flexível, pipeline de agregação nativo, índices B-Tree compostos e sharding automático.

O MongoDB vence porque resolve exactamente os nossos problemas: atributos variáveis por categoria, busca facetada eficiente e escalabilidade horizontal. Passamos à modelação. Egas?

Query-Driven Design — Estrutura do Documento

■ EGAS MANUEL RIBEIRO

Obrigado, Bengui.

Com a ferramenta escolhida, vamos ver como modelámos os dados. O princípio fundamental é o Query-Driven Design — o modelo serve os padrões de acesso, não o contrário.

No lado esquerdo, podem ver um exemplo real de documento MongoDB para um smartphone Samsung. Cada documento tem campos obrigatórios — SKU, nome, categoria, preço — e um campo especial chamado `atributos_dinamicos`.

É aqui que está o poder do MongoDB. Para eletrónicos, `atributos_dinamicos` contém RAM, armazenamento, câmara. Para vestuário, contém tamanho, cor, material. Documentos da mesma coleção com estruturas completamente diferentes — sem schema rígido.

Usámos dois padrões de modelação: o Padrão Embedding — dados que sempre são necessários juntos ficam embutidos no mesmo documento. Atributos dinâmicos, imagens, avaliação resumida — tudo num único documento. Isto elimina completamente os JOINS e optimiza a leitura.

E o Padrão Referência — dados que crescem ilimitadamente ficam em coleção separada. Reviews — um produto pode ter mil avaliações — e dados de fornecedores ficam referenciados. Isto evita o fenómeno de array explosion, onde documentos crescem indefinidamente e degradam a performance.

Índices Otimizados — Estratégia ESR

■ EGAS MANUEL RIBEIRO (continua)

O modelo de dados resolve metade do problema. A outra metade são os índices.

Utilizámos a estratégia ESR — Equality, Sort, Range — que define a ordem óptima dos campos num índice composto.

Primeiro: Equality — campos de igualdade exacta. No nosso caso, categoria = 'Eletrônicos'. O MongoDB elimina imediatamente todos os documentos que não pertençam a esta categoria.

Segundo: Sort — campos de ordenação. Ordenamos por rating_medio descendente para mostrar os melhores produtos primeiro.

Terceiro: Range — campos de intervalo. Filtramos por preço entre 5.000 e 50.000 AOA.

O resultado? 145 milissegundos contra 2.000 milissegundos sem índice — uma melhoria de 15 a 35 vezes.

No total, criámos 10 índices estratégicos: desde o índice único por SKU, passando pelo índice de texto completo para pesquisa textual, até ao índice compound que serve a busca facetada. Cada índice foi desenhado para um padrão de acesso específico.

SLIDE 7 — IMPLEMENTAÇÃO

Ambiente, Dados e Arquitectura Docker

■ MATONDO DOMINGOS BUNGA

Obrigado, Egas. Agora vamos à implementação prática.

O nosso ambiente é totalmente reproduzível com Docker. Configurámos um Replica Set de 3 nós MongoDB — um nó Primary que recebe todas as escritas, e dois nós Secondary que mantêm réplicas dos dados. Se o Primary cair, a eleição automática promove um Secondary em menos de 15 segundos.

O stack tecnológico é: MongoDB 7.0 em contentores Docker, Python 3.9 para geração de dados com a biblioteca Faker, e mongosh para execução das queries.

Para o seeding dos dados, criámos um script Python que gera 100.000 documentos realistas distribuídos em 5 categorias: 30.000 produtos de Vestuário, 20.000 de Eletrônicos, 20.000 de Casa e Cozinha, 15.000 de Livros, e 15.000 de Desportos.

Cada produto tem preços realistas, avaliações, stock e atributos específicos da sua categoria. A taxa de inserção foi de aproximadamente 33.000 documentos por segundo, completando os 100 mil documentos em cerca de 3 minutos.

A reprodutibilidade é garantida — qualquer pessoa pode clonar o repositório, executar docker-compose up, e ter o ambiente completamente funcional.

6 Queries Avançadas — Resultados de Latência

■ MATONDO DOMINGOS BUNGA (continua)

Com os dados carregados, implementámos 6 queries avançadas para demonstrar o poder do MongoDB. Cada query foi cronometrada.

Q1 — Busca Facetada: A query mais importante, representando 60% do tráfego. Filtra eletrônicos Samsung em stock, entre 10.000 e 50.000 AOA, ordenados por rating. Utiliza o índice compound ESR. Resultado: 145 milissegundos.

Q2 — Agregação por Categoria: Utiliza o pipeline de agregação com \$group e \$sort para calcular estatísticas por categoria. 235 milissegundos.

Q3 — Full-Text Search: Pesquisa textual por 'samsung smartphone' utilizando o índice de texto. 78 milissegundos — a mais rápida das queries complexas.

Q4 — Atributos Dinâmicos: Filtra dentro do subdocumento atributos_dinamicos por marca, RAM mínima e armazenamento. 102 milissegundos.

Q5 — Análise de Rentabilidade: Calcula margens de lucro por categoria com cálculos derivados. 189 milissegundos.

Q6 — Recomendações: Encontra produtos similares com preço dentro de mais ou menos 20% e mesma categoria. 68 milissegundos.

Todas as queries ficaram dentro do nosso objectivo de P99 inferior a 500ms. Passamos ao Hermenegildo para a discussão crítica.

Discussão Crítica

■ HERMENEGILDO WILSON DOS SANTOS PANZO

Obrigado, Matondo.

Uma boa solução não é só funcionar — é resistir ao crescimento e às falhas. Vamos discutir dois cenários críticos.

Cenário 1: Crescimento 100 vezes. Actualmente temos 100.000 produtos e suportamos 1.000 pedidos por segundo. Se crescermos 100 vezes — 10 milhões de produtos e 100.000 pedidos por segundo — o que acontece?

O MongoDB responde com sharding automático. A shard key que definimos é composta por categoria e preço. O MongoDB distribui automaticamente os dados entre múltiplos shards. Com 10 shards, o throughput escala linearmente e a latência mantém-se constante nos 100 a 200 milissegundos.

Há limitações a reconhecer: a shard key é imutável após a criação, e queries que não incluam a shard key visitam todos os shards. São decisões que temos de tomar com cuidado antes de ir para produção.

Cenário 2: Tolerância a Partições de rede. Se o nó Primary cair: os nós detectam a falha; eleição automática em 10 a 15 segundos; um Secondary é promovido a Primary; as escritas continuam sem interrupção manual; e quando o nó original recuperar, sincroniza automaticamente.

Os números finais: downtime de aproximadamente 15 segundos, zero dados perdidos com write concern majority, RTO inferior a 30 segundos, RPO inferior a 5 segundos.

Resultados Alcançados

■ HERMENEGILDO WILSON DOS SANTOS PANZO (conclui)

Para concluir, o nosso trabalho demonstrou que a escolha correcta de um SGBD NoSQL — fundamentada no Teorema CAP e nos padrões de acesso reais — faz toda a diferença em cenários de alta escala.

Os resultados alcançados foram: justificação rigorosa do MongoDB como SGBD AP; desnormalização estratégica com embedding e referência correctamente aplicados; busca facetada em 145 milissegundos contra 2 a 5 segundos num SGBD relacional equivalente; failover automático em menos de 15 segundos sem perda de dados; arquitectura preparada para escalar 100 vezes via sharding automático; e 100.000 documentos realistas em 5 categorias com 10 índices otimizados.

Como o nosso Docente afirma no enunciado: 'A excelência na engenharia de dados não reside apenas em conhecer as ferramentas, mas em saber exactamente quando e porquê utilizá-las.' Acreditamos ter demonstrado exactamente isso.

Agradecemos a atenção. Ficamos ao dispor para quaisquer questões.

DISTRIBUIÇÃO DE TEMPO

Apresentador	Slides	Duração
Américo Malungo	1 – 2 (Capa + Agenda)	~3 min
Bengui Bena	3 – 4 (Domínio + CAP)	~6 min
Egas Manuel	5 – 6 (Modelação + ESR)	~5 min
Matondo Domingos	7 – 8 (Implementação + Queries)	~6 min
Hermenegildo Wilson	9 – 10 (Discussão + Conclusão)	~5 min
TOTAL		~25 min

POSSÍVEIS QUESTÕES DO DOCENTE

P: Porque não usaram Cassandra, que tem latência mais baixa?

R: Cassandra tem schema rígido — não consegue lidar nativamente com atributos dinâmicos por categoria sem colunas NULL massivas ou múltiplas tabelas. O MongoDB resolve este problema de forma nativa com documentos flexíveis.

P: O que é consistência eventual na prática?

R: Num replica set MongoDB, uma escrita no Primary pode demorar alguns milissegundos a propagar para os Secondary. Nesse intervalo, uma leitura num Secondary pode retornar um valor ligeiramente desatualizado. Para um catálogo de produtos, isto é aceitável — se o preço ficar desatualizado por 2 segundos, não causa dano ao negócio.

P: E se a shard key for mal escolhida?

R: Uma má shard key — por exemplo, um campo de baixa cardinalidade — cria hotspots, sobrecarregando um único shard enquanto os outros ficam ociosos. A nossa escolha de {categoria, preço} distribui bem a carga porque categoria tem 5 valores distintos e preço tem alta cardinalidade.

P: Como garantem que os dados gerados são realistas?

R: Usámos a biblioteca Python Faker com distribuições realistas de preços por categoria, avaliações numa curva normal centrada em 4.0, e stocks com distribuição que reflecte padrões reais de inventário.

P: O embedding não pode causar documentos demasiado grandes?

R: Sim, por isso usámos referência para reviews e fornecedores. O MongoDB tem um limite de 16MB por documento. Com atributos_dinamicos de 10 a 15 campos e imagens como URLs (não binários), os nossos documentos ficam muito abaixo desse limite.

"A excelência na engenharia de dados não reside apenas em conhecer as ferramentas, mas em saber exactamente quando e porquê utilizá-las."

— Moyo Kanivengidio, Enunciado SGBD II 2025/2026